ULB UNIVERSITÉ LIBRE DE BRUXELLES

EPFC

PRM2
Théorie : Leçon 15

Java :
La récursivité

La récursivité

Une notion est récursive si elle se contient elle-même en partie ou si elle est partiellement définie à partir d'elle-même.

Elle nécessite toujours au moins deux parties :

Le cas de base : cas simple défini en ne faisant pas appel à la notion que l'on est en train de définir.

L'étape récursive : on définit la notion en fonction d'elle-même mais sur une taille plus petite.

Exemple : factorielle

Définition itérative :

`fact(n)` = produit des nombres de 1 à n *(n > 0)*.

Définition récursive :

`fact(1) = 1`

Cas de base où la définition ne fait pas apparaitre `fact`.

`fact (n) = n * fact(n-1)` *(n > 1)*

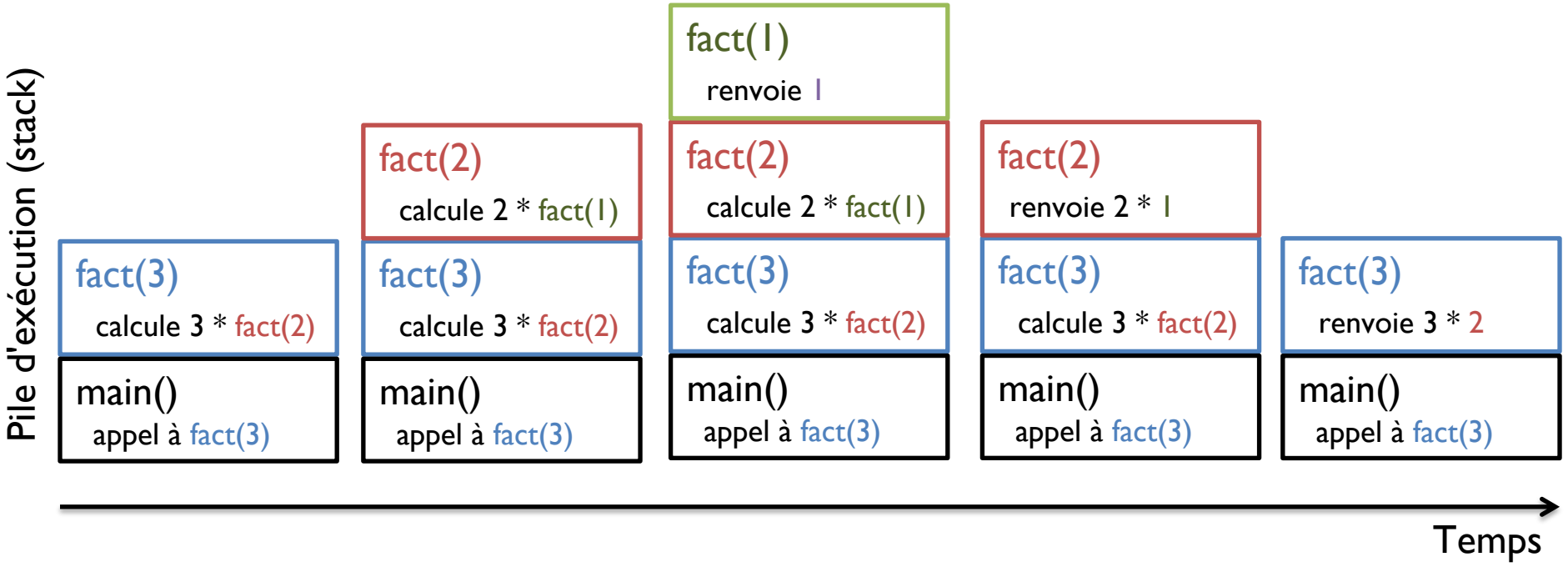
Etape récursive où `fact(n)` est défini à partir de `fact(n-1)` qui est plus "petit".

Dans cette définition, le fait que `fact(n-1)` soit plus petit signifie ici que l'argument `(n-1)` est plus petit et donc, de proche en proche, on arrive forcement au cas de base, ce qui est primordial.

Exemple : pile d'exécution


```
public static int fact(int n){
  if(n <= 1)
    return 1;
  else
    return n * fact(n-1);
}
```

```
public static void main(…){
  System.out.println(fact(3));
}
```



La recherche dichotomique (support à l'exercice 9)

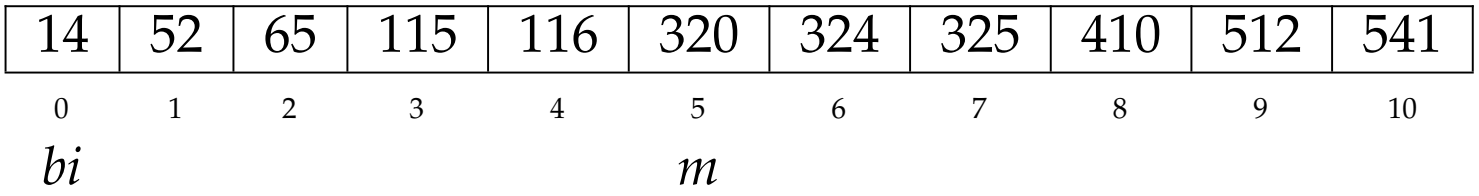
Recherche dichotomique (1)

La recherche dichotomique est un algorithme efficace pour trouver un élément dans une séquence triée.

Pour trouver une valeur x dans un intervalle trié $[bi, bs[$ (bi inclus, bs non-inclus) :

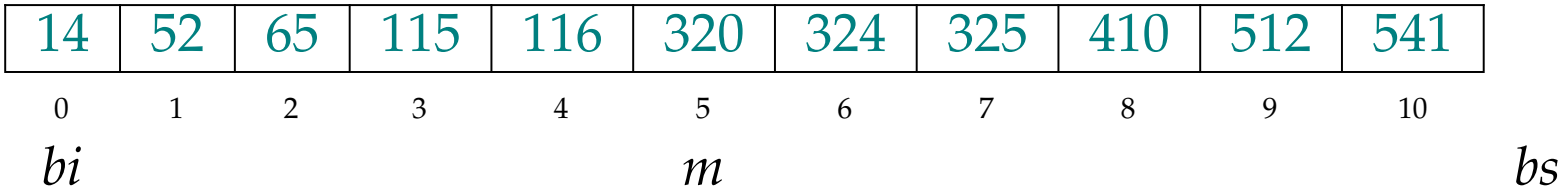
- On compare x avec la valeur m au milieu de l'intervalle $(bi + bs)/2$.
- Si la recherche n'est pas fructueuse, on réitère la recherche avec la première partie $[bi, m[$ de l'intervalle ou la seconde $[m + 1, bs[$ suivant l'endroit où x est potentiellement présent.

Remarque : si bi est égal à bs, l'intervalle de recherche est vide.

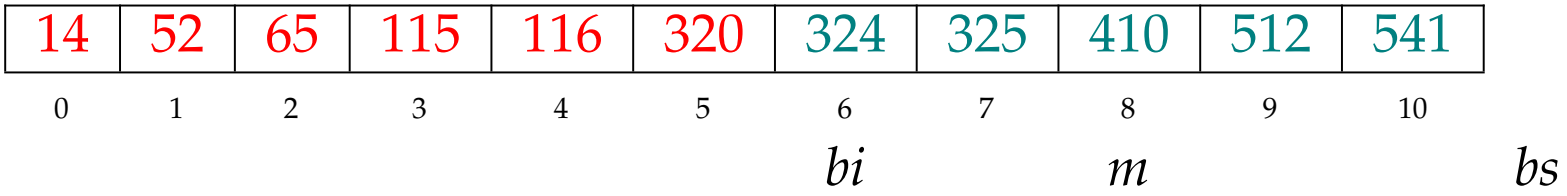


Recherche dichotomique (2)

Recherche de 324.

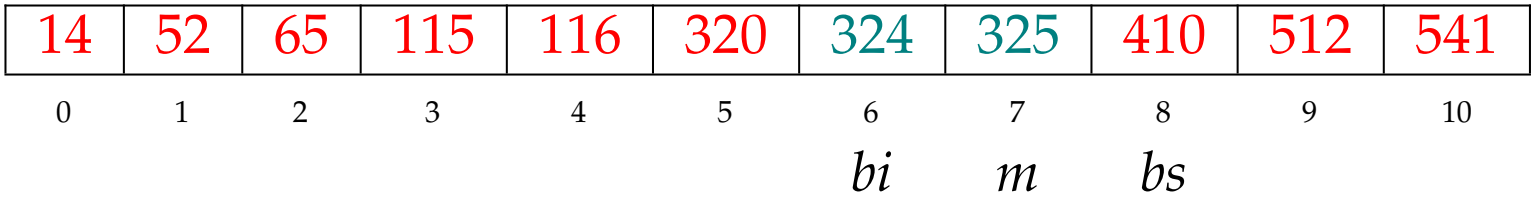


324 est supérieur à 320 : recherche à droite

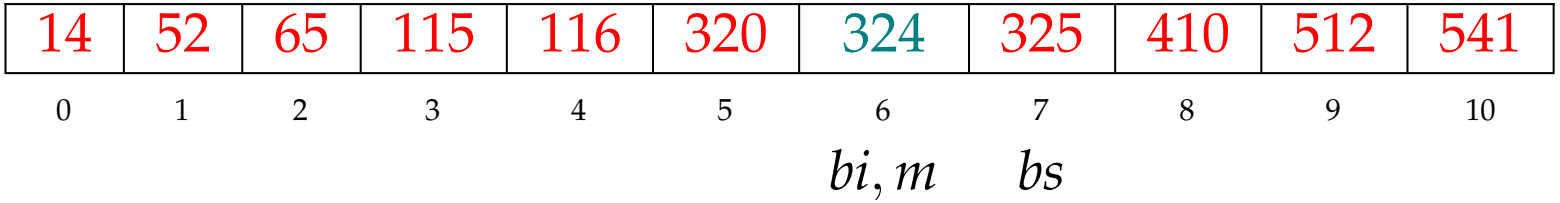


324 est inférieur à 410 : recherche à gauche

Recherche dichotomique (3)

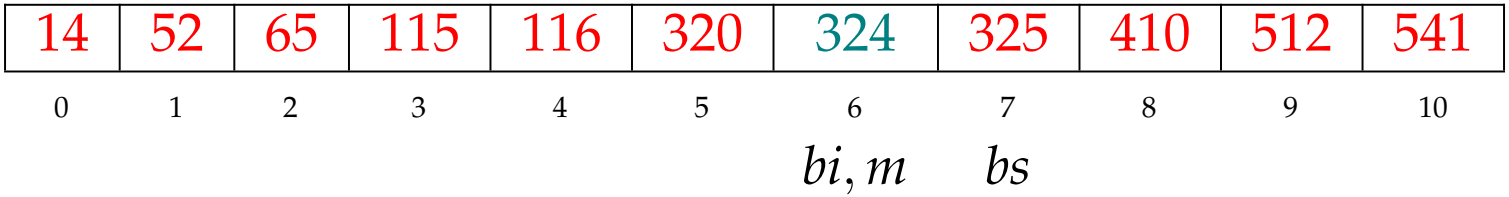


324 est inférieur à 325 : recherche à gauche



On a trouvé l'élément 324.

Recherche dichotomique (4)



Si *bi* devient égal à *bs*, alors l'intervalle de recherche est vide, ce qui signifie que l'élément cherché n'est pas dans la séquence.

Remarquez que si l'on ne trouve pas l'élément, à la fin de l'algorithme, *m* est la position où devrait se trouver cet élément.

L'algorithme est efficace car au lieu de tester chaque élément (n opérations dans le pire des cas), on réduit à chaque fois l'espace de recherche par deux. On fera donc grosso modo $\log_2(n)$ opérations au pire cas.

Recherche dichotomique (itérative) en Java

```
public static boolean recherche(Array<Integer> v, int val) {

    int bi = 0, bs = v.size(), m = (bi + bs) / 2;

    while(bi < bs && v.get(m) != val) {

        if(val < v.get(m)) //On cherche à gauche
            bs = m;
        else               //On cherche à droite
            bi = m + 1;

        m = (bi + bs) / 2;
    }

    return bi < bs;
}
```